



ORGANIZACIÓN DEL COMPUTADOR
Cátedra Benítez

Trabajo Práctico

Ingeniería Inversa Sobre Bomba con ptrace

✕ ✕

15 de junio de 2026

Agliani, Ignacio

114276

Índice

1. Introducción	3
2. Análisis del ejecutable	3
2.1. Herramientas y metodología	3
2.2. Información general	4
2.3. Convención de llamada en x86-64	4
2.4. Flujo de ejecución	4
2.5. Las seis fases	4
2.6. Análisis detallado por fase	5
3. Conceptos fundamentales	7
3.1. Formato ELF y ejecutables PIE	7
3.2. El uso de <code>ptrace</code>	8
3.3. SIGTRAP y breakpoints de software	8
4. Implementación del desactivador	9
4.1. Estrategia	9
4.2. Arquitectura general	9
4.3. Función <code>obtener_direccion_bomba</code>	9
4.4. Funciones <code>colocar_breakpoint</code> y <code>quitar_breakpoint</code>	10
4.5. Funciones <code>leer_direccion_memoria</code> e <code>inyectar</code>	11
4.6. Función principal	12
4.7. Loop de procesamiento y manejo de señales	13
4.8. Inyección por tipo de fase	14
5. Compilación y ejecución	15
6. Resultados	15
7. Conclusión	15

1. Introducción

El *BombLab* es un ejercicio clásico de ingeniería inversa. El binario solicita seis claves al usuario. Si alguna es incorrecta, la “bomba explota” y el proceso termina con error. En la versión tradicional las claves están codificadas en el binario y pueden encontrarse mediante análisis estático del código ensamblador.

La variante presentada en este trabajo introduce una diferencia fundamental: **las claves no están almacenadas en el ejecutable sino que se generan aleatoriamente en cada ejecución**. Esto hace que el análisis estático sea insuficiente: no existe ninguna clave fija que pueda extraerse del binario con antelación.

El objetivo del trabajo es escribir un programa en C que:

1. Ejecute el binario *Bomba* como proceso hijo.
2. Utilice `ptrace` para interceptar su ejecución en tiempo real.
3. Lea el valor esperado en cada fase directamente desde los registros o la memoria del proceso.
4. Inyecte ese valor en el operando que representa la respuesta del usuario, justo antes de que se evalúe la comparación.
5. Logre pasar las seis fases sin que la bomba detone.

Este informe detalla los conceptos teóricos involucrados, el análisis del ejecutable mediante ingeniería inversa y el diseño del programa `desactivador.c`.

2. Análisis del ejecutable

2.1. Herramientas y metodología

El análisis del binario se realizó en dos etapas:

1. **Exploración con editor hexadecimal:** Se utilizó la herramienta web `hexed.it` para inspeccionar el contenido del binario. Esto permitió identificar el tipo de dato de la clave de cada fase.

```
Bomba x
00001FF0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00002000 01 00 02 00 0F 12 03 4A 42 4F 4F 4D 21 20 46 61 .....JB00M! Fa
00002010 6C 6C 61 73 74 65 20 75 6E 61 20 66 61 73 65 2E llaste una fase.
00002020 00 25 64 00 25 66 00 0A 00 42 6F 6D 62 61 20 64 .%d.%f...Bomba d
00002030 65 73 61 63 74 69 76 61 64 61 20 63 6F 6E 20 45 esactivada con E
00002040 78 69 74 6F 21 00 00 00 00 96 20 44 65 73 61 63 xito!...u Desac
00002050 74 69 76 61 6E 64 6F 20 62 6F 6D 62 61 20 70 65 tivando bomba pe
00002060 72 73 6F 6E 61 6C 69 7A 61 64 61 2E 2E 2E 00 00 rsonalizada.....
00002070 00 00 00 00 00 00 00 00 46 61 73 65 20 31 3A 20 .....Fase 1:
00002080 49 6E 67 72 65 73 61 20 65 6C 20 63 6F 64 69 67 Ingresa el codig
00002090 6F 20 73 65 63 72 65 74 6F 20 28 65 6E 74 65 72 o secreto (enter
000020A0 6F 29 20 3E 20 00 00 00 46 61 73 65 20 32 3A 20 o) > ...Fase 2:
000020B0 49 6E 67 72 65 73 61 20 65 6C 20 63 6F 64 69 67 Ingresa el codig
000020C0 6F 20 73 65 63 72 65 74 6F 20 28 66 6C 6F 61 74 o secreto (float
000020D0 29 20 3E 20 00 00 00 00 46 61 73 65 20 33 3A 20 ) > ....Fase 3:
000020E0 49 6E 67 72 65 73 61 20 65 6C 20 63 6F 64 69 67 Ingresa el codig
000020F0 6F 20 73 65 63 72 65 74 6F 20 28 65 6E 74 65 72 o secreto (enter
00002100 6F 29 20 3E 20 00 00 00 46 61 73 65 20 34 3A 20 o) > ...Fase 4:
00002110 49 6E 67 72 65 73 61 20 65 6C 20 63 6F 64 69 67 Ingresa el codig
00002120 6F 20 73 65 63 72 65 74 6F 20 28 74 65 78 74 6F o secreto (texto
00002130 29 20 3E 20 00 00 00 00 46 61 73 65 20 35 3A 20 ) > ....Fase 5:
00002140 49 6E 67 72 65 73 61 20 65 6C 20 63 6F 64 69 67 Ingresa el codig
00002150 6F 20 73 65 63 72 65 74 6F 20 28 65 6E 74 65 72 o secreto (enter
00002160 6F 29 20 3E 20 00 00 00 46 61 73 65 20 36 3A 20 o) > ...Fase 6:
00002170 49 6E 67 72 65 73 61 20 65 6C 20 63 6F 64 69 67 Ingresa el codig
00002180 6F 20 73 65 63 72 65 74 6F 20 28 66 6C 6F 61 74 o secreto (float
00002190 29 20 3E 20 00 00 00 00 21 21 21 21 21 21 21 ) > .....!!!!!!
```

2. **Desensamblado con objdump:**

```
objdump -d -M intel Bomba > desensamblado.txt
```

El análisis sobre el código desensamblado consistió en:

- Localizar las instrucciones de comparación (`cmp`, `ucomiss`, `call strcmp`) de cada fase.

- Rastrear **hacia atrás** desde cada comparación para encontrar en qué registro o posición del stack quedaba almacenada la clave generada aleatoriamente o, en otros casos, si se sabe en qué registro está lo ingresado por el usuario, alcanza con analizar las instrucciones de comparación.
- Rastrear la llamada a `sscanf` anterior para identificar, mediante la convención de llamada, en qué dirección de memoria quedaba el input del usuario.

2.2. Información general

```
$ file Bomba
Bomba: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=2ec7e50a..., for GNU/Linux 3.2.0, stripped
```

Características relevantes:

- **Arquitectura:** x86-64 (64 bits, little-endian).
- **Tipo:** ELF PIE, enlazado dinámicamente.

2.3. Convención de llamada en x86-64

En la arquitectura x86-64, los primeros seis argumentos de una función se pasan en los registros: `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`. El valor de retorno queda en `rax`.

Conocer esta convención es esencial para el análisis del desensamblado. En particular, para las dos funciones que procesan el input del usuario:

- `fgets(buffer, tamaño_buffer, archivo_entrada)`: `rdi` = buffer donde queda el input del usuario; `rsi` = tamaño máximo en bytes; `rdx` = archivo de entrada (`stdin`).
- `sscanf(str_a_convertir, formato, &destino)`: el valor convertido queda en la dirección apuntada por `rdx`. Observando el valor de `rdx` antes de la llamada se sabe exactamente en qué posición del stack deposita `sscanf` la respuesta del usuario.

2.4. Flujo de ejecución

Al iniciarse, la función `main` del binario sigue este patrón:

1. Se generan claves al azar.
2. Se guardan en registros específicos.
3. Para cada fase: imprime un mensaje solicitando un ingreso, lee la respuesta del usuario con `fgets` + `sscanf` (o `fgets` + `strcmp` para la fase de texto), compara con la clave esperada y, si no coincide, llama a la función que imprime el mensaje de explosión y termina.

2.5. Las seis fases

El análisis del desensamblado permitió identificar las instrucciones de comparación de cada fase y los registros involucrados:

Fase	Tipo	Offset ELF	Instrucción de comparación	Clave en
1	Entero	0x176e	<code>cmp [rsp+0x44], r15d</code>	registro <code>r15d</code>
2	Float	0x17c8	<code>ucomiss xmm3, [rsp+0x48]</code>	memoria <code>[rsp+0x1c]</code>
3	Entero	0x182f	<code>cmp [rsp+0x44], ebx</code>	registro <code>ebx</code>
4	Texto	0x18ae	<code>call strcmp(rdi, rsi)</code>	<code>rsi</code> → string en <code>[rsp+0x50]</code>
5	Entero	0x1911	<code>cmp [rsp+0x44], ebx</code>	registro <code>ebx</code>
6	Float	0x195f	<code>ucomiss xmm4, [rsp+0x4c]</code>	memoria <code>[rsp+0x3c]</code>

2.6. Análisis detallado por fase

Fases 1, 3 y 5 — Enteros

Para la Fase 1, la instrucción de comparación es:

```
1 176e: 44 39 7c 24 44    cmp    DWORD PTR [rsp+0x44], r15d
2 1773: 0f 85 22 02 00    jne    199b    ; B00M! si no coincide
```

Codigo 1: Comparacion de entero — Fase 1 (offset 0x176e)

Para saber por qué la clave está en `r15d`, se rastreó hacia atrás en el desensamblado:

```
1 11f0: e8 9b ff ff ff    call   rand
2 11f5: 41 89 c7          mov    r15d, eax
3 ...
4 1747: 4c 8d 74 24 44    lea    r14, [rsp+0x44]
5 174c: e8 df f9 ff ff    call   fgets
6 ...
7 1753: 4c 89 f2          mov    rdx, r14
8 1760: e8 fb f9 ff ff    call   sscanff
```

Codigo 2: Cadena de generacion de clave — Fase 1

Se observa que `rand()` devuelve un entero aleatorio en `eax`, que se copia inmediatamente a `r15d`. Por otro lado, antes de llamar a `fgets` se carga en `r14` la dirección efectiva de `[rsp+0x44]`. Luego, al llamar a `sscanf`, ese mismo valor se pasa como tercer argumento en `rdx` — que, según la convención de llamada, es el destino del valor convertido. Por lo tanto, `[rsp+0x44]` contiene la respuesta del usuario y debe sobrescribirse con el valor de `r15d`.

Las Fases 3 y 5 siguen la misma estructura, con la diferencia de que la clave queda en `ebx`. Dado que el registro `r14` no se modifica entre fases, el input del usuario sigue llegando a `[rsp+0x44]`.

```
1 ; Fase 3
2 182f: 39 5c 24 44      cmp    DWORD PTR [rsp+0x44], ebx
3 1833: 0f 85 62 01 00 00 jne    199b
4 ...
5 ; Fase 5
6 1911: 39 5c 24 44      cmp    DWORD PTR [rsp+0x44], ebx
7 1915: 0f 85 80 00 00 00 jne    199b
```

Codigo 3: Fase 3 (offset 0x182f) y Fase 5 (offset 0x1911)

Volviendo a la comparación, si vamos a `199b`, nos encontramos:

```
1 199b: 31 c0            xor    eax, eax
2 199d: e8 fe 00 00 00    call   1aa0
```

Codigo 4: Código en 199b

Es decir, se limpia el contenido del registro `eax` y realiza una `call` a `1aa0`. Yendo a `1aa0`, encontramos:

```
1 1aa0: f3 0f 1e fa      endbr64
2 1aa4: 50               push   rax
3 1aa5: 58               pop    rax
4 1aa6: 48 8d 3d 5b 05 00 00 lea    rdi, [rip+0x55b]
5 1aad: 48 83 ec 08      sub    rsp, 0x8
6 1ab1: e8 3a f6 ff ff    call   puts
7 1ab6: bf 01 00 00 00    mov    edi, 0x1
8 1abb: e8 c0 f6 ff ff    call   exit
```

Codigo 5: Código en 1aa0

Es importante notar que carga en el `RDI` una dirección de memoria, hace un llamado a la función `puts` y llama a una función `exit`. Dicho de otro modo, se puede deducir que imprime el mensaje explosión de bomba y termina el programa.

Fases 2 y 6 — Punto flotante

En la Fase 2, la comparación se realiza con la instrucción `ucomiss` (*Unordered Compare Scalar Single-Precision*):

```
1 17c2: f3 0f 10 5c 24 1c movss xmm3, [rsp+0x1c]
2 17c8: 0f 2e 5c 24 48 ucomiss xmm3, [rsp+0x48]
3 17cd: 0f 8a c8 01 00 00 jp 199b
4 17d3: 0f 85 c2 01 00 00 jne 199b
```

Código 6: Comparación de float — Fase 2 (offset 0x17c2)

La instrucción `ucomiss` compara dos valores en precisión simple (float de 32 bits). Activa el *parity flag* (indicador de paridad: bit del registro de flags que se activa cuando algún operando es un valor NaN) si alguno de los operandos no es un número válido. Las dos ramas condicionales (`jp` y `jne`) garantizan que tanto NaN como valores distintos detonen la bomba.

La generación del float se rastreó hacia atrás:

```
1 11f8: e8 93 ff ff ff call rand
2 11fd: 66 0f ef c0 pxor xmm0, xmm0
3 1201: f3 0f 2a c0 cvtsi2ss xmm0, eax
4 1205: f3 0f 5e 05 f7 0d divss xmm0, [rip+0xdf7]
5 120d: f3 0f 11 44 24 1c movss [rsp+0x1c], xmm0
```

Código 7: Cadena de generación del float — Fase 2

Primero, `rand()` genera un entero en `eax`. Luego, `pxor xmm0, xmm0` inicializa el registro en cero. La instrucción `cvtsi2ss` (*Convert Scalar Integer to Scalar Single*) convierte el entero de `eax` a un float de 32 bits en `xmm0`. A continuación, `divss` (*Divide Scalar Single-precision*) lo divide por una constante almacenada en el segmento de datos del binario. El resultado se guarda en `[rsp+0x1c]` con `movss` (*Move Scalar Single Precision*). El input del usuario, procesado por `scanf` con formato `%f`, se deposita en `[rsp+0x48]`.

La Fase 6 sigue el mismo patrón con el registro `xmm4` y los offsets `[rsp+0x3c]` (clave) y `[rsp+0x4c]` (input del usuario).

```
1 1959: f3 0f 10 64 24 3c movss xmm4, DWORD PTR [rsp+0x3c]
2 195f: 0f 2e 64 24 4c ucomiss xmm4, DWORD PTR [rsp+0x4c]
3 1964: 7a 35 jp 199b
4 1966: 75 33 jne 199b
```

Código 8: Comparación de float — Fase 6 (offset 0x195f)

Fase 4 — Texto

La Fase 4 pide al usuario que ingrese un string. La comparación se hace con `strcmp`:

```
1 187e: 48 8b 15 8b 27 00 00 mov rdx, QWORD PTR [rip+0x278b] # 4010 <
    stdin@GLIBC_2.2.5>
2 1885: be 64 00 00 00 mov esi, 0x64
3 188a: 48 89 ef mov rdi, rbp
4 188d: e8 9e f8 ff ff call fgets
5 1892: 48 8d 35 8e 07 00 00 lea rsi, [rip+0x78e]
6 1899: 48 89 ef mov rdi, rbp
7 189c: e8 6f f8 ff ff call strchr
8 18a1: 48 8d 74 24 50 lea rsi, [rsp+0x50]
9 18a6: 48 89 ef mov rdi, rbp
10 18a9: c6 44 04 60 00 mov BYTE [rsp+rax+0x60], 0x0
11 18ae: e8 8d f8 ff ff call strcmp
12 18b3: 85 c0 test eax, eax
13 18b5: 0f 85 e0 00 00 00 jne 199b
```

Código 9: Ingreso del usuario, eliminación del newline y comparación de string — Fase 4

Se empieza pidiéndole al usuario una cadena con la función `fgets`. Considerar que el `rdi` apunta al buffer (donde va a quedar el dato), en `esi` va el tamaño del buffer (`0x64` en base 16, lo cual es 100 en base 10) y en `rdx` se carga un puntero a la entrada estándar. Antes de llamar a `strcmp`, la bomba elimina el carácter de nueva línea que `fgets` deja al final del buffer. Para esto usa `strcspn(s1, s2)`, que devuelve en `rax` el índice de la primera ocurrencia de cualquier carácter de `s2` en `s1`. En este caso, busca el carácter `'\n'` en el buffer del usuario y coloca un 0 en esa posición para terminarlo.

En el momento de `call strcmp`: `rdi` apunta al buffer del usuario y `rsi` apunta al string esperado en `[rsp+0x50]`. La instrucción `test eax, eax` activa el *zero flag* si `eax` es cero, es decir, si las cadenas son iguales.

3. Conceptos fundamentales

3.1. Formato ELF y ejecutables PIE

Los programas en Linux se almacenan en formato **ELF** (*Executable and Linkable Format*). Un archivo ELF contiene uno o más **segmentos de carga** (*load segments*): regiones del archivo que el sistema operativo copia directamente en la memoria virtual del proceso al ejecutarlo. Los segmentos de carga contienen el código ejecutable, los datos inicializados y otros elementos que el programa necesita en tiempo de ejecución.

En un ejecutable no PIE, las instrucciones de salto, los accesos a datos y las llamadas a funciones usan *direcciones absolutas* fijas, lo que obliga al sistema operativo a cargarlo siempre en la misma dirección base. Un ejecutable **PIE** (*Position Independent Executable*) usa exclusivamente *referencias relativas* al *instruction pointer* (`rip`), por lo que puede cargarse en cualquier dirección sin recompilación. Las direcciones que aparecen en el desensamblado son *offsets* respecto al inicio del ELF, no direcciones reales en memoria.

Kernel

El **kernel** (núcleo del sistema operativo) es el programa que se ejecuta con los mayores privilegios del procesador y actúa como intermediario entre los programas de usuario y el hardware. Administra la memoria, el procesador, los dispositivos y los sistemas de archivos.

`/proc/<pid>/maps`

`/proc` es un **sistema de archivos virtual** del kernel de Linux: no existe en el disco; el kernel lo genera en tiempo real cuando algún programa lo lee. Dentro de `/proc`, cada proceso tiene un directorio `/proc/<pid>/` con archivos que exponen su estado interno. El archivo `/proc/<pid>/maps` describe la distribución del espacio de direcciones virtuales del proceso: cada línea corresponde a un segmento mapeado e indica su dirección de inicio, dirección de fin, permisos de acceso, offset en el archivo fuente y, si aplica, el nombre del archivo en disco.

```
584ae0e57000-584ae0e5c000 r-xp 00000000 08:01 1234567 .../Bomba
```

Código 10: Ejemplo de línea en `/proc/<pid>/maps` para la Bomba

La primera columna (`584ae0e57000`) es la dirección base donde el kernel cargó ese segmento.

ASLR — Address Space Layout Randomization

El kernel implementa **ASLR**, que asigna una dirección base aleatoria a cada proceso en cada ejecución. Combinado con PIE, esto significa que la dirección de cualquier instrucción cambia en cada ejecución. Para colocar un breakpoint es necesario:

1. Conocer el *offset* de la instrucción en el ELF (obtenido del desensamblado).
2. Determinar la dirección base real en runtime leyendo `/proc/<pid>/maps`.
3. Calcular: `dir_real = base_carga + offset_ELF`.

3.2. El uso de ptrace

ptrace (*process trace*) es una herramienta de Linux que permite a un proceso *trazador* observar y controlar la ejecución de otro proceso *trazado*.

La firma de **ptrace** es:

```
long ptrace(enum __ptrace_request request, pid_t pid, void *addr, void *data);
```

request Operación a realizar (ver tabla a continuación).

pid Identificador del proceso trazado.

addr Dirección de memoria del trazado sobre la que operar. Se pasa **NULL** cuando la operación no requiere una dirección.

data Dato adicional según la operación: señal a entregar, puntero a estructura de registros, valor a escribir, etc. Se pasa **NULL** cuando no aplica.

Las operaciones de **ptrace** utilizadas en este trabajo son:

Operación	Descripción
PTRACE_TRACEME	El hijo se declara trazable por su padre. Los argumentos pid , addr y data son ignorados (se pasan 0, NULL , NULL).
PTRACE_CONT	Reanuda la ejecución del trazado. addr es ignorado (NULL). data es la señal a entregar; NULL significa ninguna.
PTRACE_PEEKTEXT	Lee 8 bytes de la memoria del trazado en addr . data es ignorado (NULL). Devuelve el valor leído.
PTRACE_POKETEXT	Escribe el valor data en la memoria del trazado en addr .
PTRACE_GETREGS	Copia todos los registros del trazado a la estructura apuntada por data . addr es ignorado (NULL).
PTRACE_SETREGS	Escribe en los registros del trazado los valores de la estructura apuntada por data . addr es ignorado (NULL).

3.3. SIGTRAP y breakpoints de software

SIGTRAP (*Signal Trap* — señal de trampa) es una señal que el kernel envía a un proceso cuando ocurre un evento de depuración. Al recibir **SIGTRAP**, el proceso trazado se suspende automáticamente y el trazador es notificado a través de **waitpid**.

Un **breakpoint de software** consiste en reemplazar el primer byte de una instrucción con **0xCC**, que corresponde a la instrucción **INT3** (*breakpoint interrupt*) del x86. Cuando el procesador ejecuta **INT3**, genera una interrupción que el kernel convierte en **SIGTRAP** para el proceso. Se eligió **0xCC** porque ocupa exactamente un byte, por lo que puede insertarse en cualquier instrucción sin desplazar el resto del código.

El procedimiento completo para usar un breakpoint con **ptrace** es:

1. Leer los 8 bytes originales en la dirección objetivo con **PTRACE_PEEKTEXT** y guardarlos.
2. Escribir **0xCC** en el primer byte con **PTRACE_POKETEXT**.
3. Reanudar el trazado con **PTRACE_CONT**.
4. Cuando el breakpoint se dispara, **waitpid** retorna con señal **SIGTRAP**.
5. Notar que **rip** apunta al byte *siguiente* al **0xCC** (el CPU ya avanzó el *instruction pointer*).
6. Restaurar el byte original, corregir **rip** decrementándolo en 1 y continuar.

4. Implementación del desactivador

4.1. Estrategia

La estrategia consiste en colocar breakpoints en las instrucciones de comparación de cada fase. En ese preciso momento la Bomba ya calculó la clave esperada y la tiene disponible en un registro o en el stack. El desactivador lee ese valor e inyecta en la posición donde la Bomba espera la respuesta del usuario.

Para que la Bomba tenga algo que parsear con `sscanf`, se le envían seis respuestas ficticias a través de un pipe conectado a su `stdin`. El contenido de estas respuestas no importa ya que son sobrescritas por `ptrace` antes de la comparación.

4.2. Arquitectura general

El flujo general del desactivador es:

1. Crear un pipe (canal) de comunicación entre padre e hijo.
2. Llamar `fork()` para crear un proceso hijo, idéntico al padre.
3. El proceso hijo conecta su `stdin` al pipe para que su padre le inyecte las respuestas correctas.
4. Se usa `execve` para reemplazar el programa en ejecución por uno nuevo (el de la Bomba), con el fin de acceder a stack y segmentos de memoria.
5. El padre espera el `SIGTRAP` inicial de `execve`, lee la dirección base del ELF y coloca 6 breakpoints.
6. El padre escribe las respuestas ficticias en el pipe.
7. El padre procesa los 6 breakpoints en orden: reanuda al hijo, espera el `SIGTRAP`, lee registros, inyecta la clave, restaura la instrucción y corrige `rip`.
8. Reanuda al hijo hasta que termina.

Las siguientes secciones describen cada función del programa. Las funciones del sistema utilizadas se describen junto a su primera aparición.

4.3. Función `obtener_direccion_bomba`

Recibe el PID del proceso hijo y devuelve la dirección base donde el sistema operativo cargó el ejecutable.

Para determinar esa dirección se lee el archivo virtual `/proc/<pid>/maps`, que el kernel mantiene por cada proceso en ejecución con la lista de todos sus segmentos de memoria: el ejecutable, las librerías dinámicas, el stack, el heap, etc. Se busca la línea que contiene el nombre "Bomba" y se extrae la dirección de inicio.

Funciones del sistema utilizadas:

`snprintf(buffer, tamaño_buffer, formato, ...)` Escribe en el `buffer` una cadena formateada sin superar `tamaño_buffer` bytes. Se usa para construir la ruta `/proc/<pid>/maps`.

`fopen(pathname, modo)` Abre el archivo en `pathname` con el modo indicado ("`r`" = solo lectura). Devuelve un puntero a `FILE` o `NULL` si falló.

`fgets(buffer, tamaño_buffer, archivo)` Lee hasta `tamaño_buffer-1` caracteres del archivo `archivo` y los guarda en `buffer` con terminador nulo. Devuelve `NULL` al fin de archivo o si ocurrió un error.

`strstr(cadena1, cadena2)` Busca la primera ocurrencia de la cadena `cadena2` dentro de `cadena1`. Devuelve un puntero a la coincidencia o `NULL`.

sscanf(buffer, formato, ...) Parsea el string en **buffer** según **formato** y guarda los resultados en las direcciones adicionales. Se usa con **%lx** para extraer la dirección base en hexadecimal.

fclose(archivo) Cierra el archivo representado por **archivo**.

```
1 unsigned long obtener_direccion_bomba(pid_t id_proceso_bomba) {
2     char path[64];
3     snprintf(path, sizeof(path), "/proc/%d/maps", (int)id_proceso_bomba);
4     FILE *procesos = fopen(path, "r");
5     if (!procesos) {
6         perror("Proceso no se Encuentra");
7         exit(1);
8     }
9     unsigned long base = 0;
10    char linea[512];
11    while (fgets(linea, sizeof(linea), procesos)) {
12        if (strstr(linea, "Bomba")) {
13            sscanf(linea, "%lx-", &base);
14            break;
15        }
16    }
17    fclose(procesos);
18    return base;
19 }
```

Codigo 11: Funcion obtener_direccion_bomba

Se declara **base = 0**. El ciclo **while** lee el archivo línea por línea con **fgets**. Por cada línea, **strstr** verifica si contiene la cadena "Bomba". Cuando la encuentra, **sscanf** extrae el número hexadecimal al inicio de la línea (la dirección base) y lo guarda en **base**. Si al terminar el ciclo **base** sigue en cero, el programa termina con error.

4.4. Funciones colocar_breakpoint y quitar_breakpoint

colocar_breakpoint recibe el PID del hijo y la dirección donde colocar el breakpoint. Devuelve los 8 bytes originales de esa dirección para poder restaurarlos después.

PTRACE_PEEKTEXT y **PTRACE_POKETEXT** operan sobre palabras de 8 bytes alineados. Para no corromper los 7 bytes restantes de la instrucción, se aplica la máscara **& ~0xFFL** que preserve todos los bytes excepto el primero, y luego se realiza **| 0xCCL** para insertar el byte **0xCC**.

Dado que **PTRACE_PEEKTEXT** puede devolver **-1** como valor legítimo de memoria, la única forma de detectar un error real es reseteando **errno** a cero antes de la llamada y verificando si cambió después.

```
1 long colocar_breakpoint(pid_t id_proceso_bomba,
2                         unsigned long direccion_breakpoint) {
3     errno = 0;
4     long bytes_originales = ptrace(PTRACE_PEEKTEXT,
5                                   id_proceso_bomba,
6                                   (void *)direccion_breakpoint,
7                                   NULL);
8     if (errno != 0) {
9         perror("No se pudo copiar los bytes de 'direccion_breakpoint'");
10        exit(1);
11    }
12    long interrupcion = (bytes_originales & ~0xFFL) | 0xCCL;
13    if (ptrace(PTRACE_POKETEXT, id_proceso_bomba,
14              (void *)direccion_breakpoint,
15              (void *)interrupcion) < 0) {
16        perror("No se pudo generar interrupcion");
17        exit(1);
18    }
```

```
18     }
19     return bytes_originales;
20 }
21
22 void quitar_breakpoint(pid_t id_proceso_bomba,
23                       unsigned long direccion_breakpoint_modificada,
24                       long bytes_originales) {
25     if (ptrace(PTRACE_POKETEXT, id_proceso_bomba,
26               (void *)direccion_breakpoint_modificada,
27               (void *)bytes_originales) < 0) {
28         perror("No se pudieron restaurar los bytes originales");
29         exit(1);
30     }
31 }
```

Codigo 12: Funciones colocar_breakpoint y quitar_breakpoint

quitar_breakpoint recibe los mismos parámetros que colocar_breakpoint más los bytes originales guardados, y simplemente los escribe de vuelta con PTRACE_POKETEXT.

4.5. Funciones leer_direccion_memoria e inyectar

Las claves de tipo entero y float ocupan 4 bytes (32 bits). Sin embargo, PTRACE_PEEKTEXT y PTRACE_POKETEXT siempre operan con palabras de 8 bytes alineadas a 8. Las funciones leer_direccion_memoria e inyectar resuelven este desajuste.

leer_direccion_memoria recibe el PID y una dirección, y devuelve los 4 bytes almacenados en esa dirección. Primero calcula la dirección alineada ($\text{addr} \& \sim 7\text{UL}$), lee los 8 bytes completos, calcula el desplazamiento en bits ($(\text{addr} \& 7) * 8$) y desplaza la palabra a la derecha para que los 4 bytes de interés queden en la parte baja, donde el cast a `uint32_t` los extrae.

inyectar recibe el PID, una dirección y un valor de 32 bits a escribir. Lee primero los 8 bytes existentes para no perder los 4 bytes que no se quieren modificar. Luego aplica una máscara para limpiar los 4 bytes de destino y un OR con el valor desplazado para escribirlos.

```
1  uint32_t leer_direccion_memoria(pid_t id_proceso_bomba,
2                                unsigned long direccion_a_leer) {
3      unsigned long alineado = direccion_a_leer & ~7UL;
4      errno = 0;
5      long leido = ptrace(PTRACE_PEEKTEXT, id_proceso_bomba,
6                          (void *)alineado, NULL);
7      if (errno != 0) {
8          perror("No se pudo leer la memoria de interes");
9          exit(1);
10     }
11     int desplazamiento = (int)((direccion_a_leer & 7) * 8);
12     return (uint32_t)((uint64_t)leido >> desplazamiento);
13 }
14
15 void inyectar(pid_t id_proceso_bomba,
16               unsigned long direccion_a_escribir,
17               uint32_t contenido_a_escribir) {
18     unsigned long alineado = direccion_a_escribir & ~7UL;
19     errno = 0;
20     uint64_t leido = (uint64_t)ptrace(PTRACE_PEEKTEXT, id_proceso_bomba,
21                                       (void *)alineado, NULL);
22     if (errno != 0) {
23         perror("No se pudo leer memoria");
24         exit(1);
25     }
26     int desplazamiento = (int)((direccion_a_escribir & 7) * 8);
27     leido = (leido & ~(0xFFFFFFFFFULL << desplazamiento))
28         | ((uint64_t)contenido_a_escribir << desplazamiento);
```

```
29     if (ptrace(PTRACE_POKETEXT, id_proceso_bomba,  
30             (void *)alineado, (void *)leido) < 0) {  
31         perror("No se pudo escribir memoria");  
32         exit(1);  
33     }  
34 }
```

Codigo 13: Funciones leer_direccion_memoria e injectar

Ejemplo: para escribir en `[rsp+0x44]`, con `rsp` alineado a 8 bytes, la palabra de 8 bytes está en `rsp+0x40` y los 4 bytes a escribir comienzan en el byte 4. Por lo tanto `desplazamiento = 32`, y la máscara limpia exactamente esos 4 bytes antes de insertar el nuevo valor.

4.6. Función principal

La función `main` orquesta todo el proceso. Las funciones del sistema utilizadas en este bloque son:

pipe(pipefd) Crea un canal de comunicación unidireccional. Llena `pipefd[0]` con el descriptor de lectura y `pipefd[1]` con el de escritura. Un `fd` es un *file descriptor*: variable que se utiliza en el código para referirse al número entero que identifica un canal de comunicación abierto por el sistema operativo.

fork() No recibe parámetros. Crea un proceso hijo idéntico al padre. Devuelve 0 al hijo, el PID del hijo al padre, o -1 si falló.

dup2(viejofd, nuevofd) Duplica el descriptor `viejofd` en `nuevofd`, cerrando `nuevofd` si ya estaba abierto. Se usa para conectar `stdin` del hijo al pipe.

execve(pathname, argv, envp) Reemplaza el proceso actual por el programa ubicado en `pathname`: el kernel descarta el código y la memoria del proceso existente y carga en su lugar el nuevo ejecutable, comenzando su ejecución desde el punto de entrada. `argv` es un array de strings con los argumentos del nuevo programa, que debe terminar en `NULL`; el primer elemento es por convención el nombre del programa. `envp` es un array de strings con las variables de entorno, también terminado en `NULL`; se pasa `NULL` para heredar el entorno del proceso padre. Si `execve` tiene éxito, nunca retorna al código que lo llamó porque ese código ya no existe (el hijo nunca retorna, el padre sí lo ejecuta). En un proceso trazado con `PTRACE_TRACEME`, el kernel genera automáticamente un `SIGTRAP` antes de ejecutar la primera instrucción del nuevo programa, dando al trazador la oportunidad de intervenir.

waitpid(pid, status, options) Espera a que el proceso `pid` cambie de estado. El resultado se guarda en `status`. Con `options = 0`, el proceso padre queda suspendido hasta que el proceso hijo se detiene o termina.

write(fd, buffer, n) Escribe `n` bytes del `buffer` al descriptor `fd`.

close(fd) Cierra el descriptor `fd`.

strlen(s) Devuelve la longitud de la cadena `s` sin el terminador nulo.

El bloque inicial de `main` crea el pipe y el proceso hijo. El orden es importante: el pipe se crea *antes* del `fork` para que el hijo herede automáticamente ambos extremos.

```
1 int canal[2];  
2 if (pipe(canal) < 0) {  
3     perror("No se pudo crear canal padre - hijo");  
4     return 1;  
5 }  
6 pid_t proceso_hijo = fork();  
7 if (proceso_hijo < 0) {  
8     perror("No se pudo crear proceso hijo");
```

```
9     return 1;  
10 }
```

Codigo 14: Creacion del pipe y del proceso hijo

Dentro del bloque del hijo (`proceso_hijo == 0`), se redirige `stdin` al extremo de lectura del pipe con `dup2`, se cierran los descriptores del pipe que ya no se necesitan, se declara el proceso trazable con `PTRACE_TRACEME` y se reemplaza el proceso por la Bomba con `execve`.

```
1 if (proceso_hijo == 0) {  
2     dup2(canal[0], STDIN_FILENO);  
3     close(canal[0]);  
4     close(canal[1]);  
5     ptrace(PTRACE_TRACEME, 0, NULL, NULL);  
6     char *av[] = { BOMB_PATH, NULL };  
7     execve(BOMB_PATH, av, NULL);  
8     perror("No se pudo reemplazar el proceso por el de la bomba");  
9     _exit(1);  
10 }
```

Codigo 15: Bloque del proceso hijo

El padre cierra el extremo de lectura del pipe (solo escribe), espera el `SIGTRAP` inicial que genera `execve` en un proceso trazado, obtiene la dirección base y coloca los 6 breakpoints. Luego escribe las seis respuestas ficticias en el pipe, seguidas de un cierre del extremo de escritura.

```
1 close(canal[0]);  
2 int info_hijo;  
3 waitpid(proceso_hijo, &info_hijo, 0);  
4 if (!WIFSTOPPED(info_hijo)) {  
5     fprintf(stderr, "El hijo no se detuvo tras execve\n");  
6     return 1;  
7 }  
8 unsigned long base = obtener_direccion_bomba(proceso_hijo);  
9 if (base == 0) {  
10     fprintf(stderr, "No se pudo determinar la direccion base\n");  
11     return 1;  
12 }  
13 unsigned long vector_direcciones_breakpoints[6] = {  
14     base + OFFSET_FASE1, base + OFFSET_FASE2, base + OFFSET_FASE3,  
15     base + OFFSET_FASE4, base + OFFSET_FASE5, base + OFFSET_FASE6,  
16 };  
17 long vector_contenido_previo[6];  
18 for (int i = 0; i < 6; i++)  
19     vector_contenido_previo[i] = colocar_breakpoint(  
20         proceso_hijo,  
21         vector_direcciones_breakpoints[i]);  
22 const char *respuestas = "0\n0.0\n0\nEstoSeSobreescribe\n0\n0.0\n";  
23 write(canal[1], respuestas, strlen(respuestas));  
24 close(canal[1]);
```

Codigo 16: Inicializacion del padre

4.7. Loop de procesamiento y manejo de señales

El loop externo itera 6 veces, una por fase. Dentro, un loop interno reanuda al hijo con `PTRACE_CONT` y espera su detención con `waitpid`. Si la señal recibida no es `SIGTRAP` (por ejemplo, una señal del linker dinámico), se la reenvía al hijo en el siguiente `PTRACE_CONT` y se continúa esperando. Solo cuando se recibe `SIGTRAP` se sale del loop interno.

```
1 struct user_regs_struct registros;  
2 for (int fase = 0; fase < 6; fase++) {
```

```

3 void *senal_hijo = NULL;
4 for (;;) {
5     if (ptrace(PTRACE_CONT, proceso_hijo, NULL, senal_hijo) < 0) {
6         perror("No se pudo continuar"); return 1;
7     }
8     waitpid(proceso_hijo, &info_hijo, 0);
9     if (WIFEXITED(info_hijo)) {
10         fprintf(stderr, "El hijo terminó en fase %d\n", fase + 1);
11         return 1;
12     }
13     if (!WIFSTOPPED(info_hijo)) {
14         fprintf(stderr, "Estado inesperado en fase %d\n", fase + 1);
15         return 1;
16     }
17     if (WSTOPSIG(info_hijo) == SIGTRAP) {
18         break;
19     }
20     senal_hijo = (void *) (long) WSTOPSIG(info_hijo);
21 }
22 ptrace(PTRACE_GETREGS, proceso_hijo, NULL, &registros);
23 unsigned long rip = registros.rip - 1;
24 unsigned long rsp = registros.rsp;
25 int indice = -1;
26 for (int i = 0; i < 6; i++)
27     if (rip == vector_direcciones_breakpoints[i])
28         { indice = i; break; }
29 if (indice < 0) { return 1; }
30 /* ... inyeccion segun la fase (ver seccion siguiente) ... */
31 quitar_breakpoint(proceso_hijo,
32     vector_direcciones_breakpoints[indice],
33     vector_contenido_previo[indice]);
34 registros.rip = rip;
35 ptrace(PTRACE_SETREGS, proceso_hijo, NULL, &registros);
36 }
37 ptrace(PTRACE_CONT, proceso_hijo, NULL, NULL);
38 waitpid(proceso_hijo, &info_hijo, 0);

```

Código 17: Loop de procesamiento de breakpoints

Al detectar el SIGTRAP, PTRACE_GETREGS copia todos los registros del hijo. Se resta 1 al valor de `rip` porque al ejecutar el 0xCC el CPU avanzó el *instruction pointer* un byte. Identificada la fase por comparación con las direcciones de los breakpoints, se inyecta la clave, se restaura la instrucción original y se corrige `rip` para que el hijo re-ejecute el `cmp` original con la clave ya inyectada. PTRACE_SETREGS aplica todos los cambios al proceso hijo.

4.8. Inyección por tipo de fase

```

1 switch (indice) {
2 case 0:
3     {
4         uint32_t clave_correcta = (uint32_t)registros.r15;
5         inyectar(proceso_hijo, rsp + 0x44, clave_correcta);
6     }
7     break;
8 case 1:
9     {
10        uint32_t clave_correcta = leer_direccion_memoria(
11            proceso_hijo, rsp + 0x1c);
12        inyectar(proceso_hijo, rsp + 0x48, clave_correcta);
13    }
14    break;

```

```
15 case 2:
16     {
17         uint32_t clave_correcta = (uint32_t)registros.rbx;
18         inyectar(proceso_hijo, rsp + 0x44, clave_correcta);
19     }
20     break;
21 case 3:
22     registros.rdi = registros.rsi;
23     break;
24 case 4:
25     {
26         uint32_t clave_correcta = (uint32_t)registros.rbx;
27         inyectar(proceso_hijo, rsp + 0x44, clave_correcta);
28     }
29     break;
30 case 5:
31     {
32         uint32_t clave_correcta = leer_direccion_memoria(proceso_hijo, rsp + 0
33             x3c);
34         inyectar(proceso_hijo, rsp + 0x4c, clave_correcta);
35     }
36     break;
37 }
```

Codigo 18: Switch de inyeccion por fase

Las llaves dentro de cada `case` son necesarias para declarar variables locales sin que sean visibles en los casos siguientes.

La Fase 4 merece una explicación especial. En lugar de copiar el string esperado al buffer del usuario, se modifica el registro `rdi` (primer argumento de `strcmp`) para que apunte a la misma dirección que `rsi` (el string esperado en `[rsp+0x50]`). Así, `strcmp` compara el string esperado consigo mismo, retorna siempre 0 y la fase pasa sin necesidad de conocer ni copiar el string.

5. Compilación y ejecución

El programa se compila con GCC:

```
gcc -o desactivador desactivador.c
```

Y se ejecuta en el mismo directorio donde se encuentra el binario `Bomba`:

```
./desactivador
```

6. Resultados

La siguiente salida corresponde a una ejecución exitosa:

```
Desactivando bomba personalizada...
Fase 1: Ingresa el codigo secreto (entero) > Fase 2: Ingresa el codigo secreto
(float) > Fase 3: Ingresa el codigo secreto (entero) > Fase 4: Ingresa el
codigo secreto (texto) > Fase 5: Ingresa el codigo secreto (entero) > Fase
6: Ingresa el codigo secreto (float) > Bomba desactivada con Exitó!
```

7. Conclusión

Se implementó exitosamente un programa en C que desactiva las seis fases de la bomba utilizando la llamada al sistema `ptrace`.

La estrategia de inyección en el punto de comparación resultó ser elegante y robusta: al actuar sobre los operandos en el preciso momento en que se evalúan, el desactivador no necesita conocer la aritmética interna del binario ni la semilla aleatoria. El proceso de ingeniería inversa se centró únicamente en identificar *dónde* está cada comparación y *qué registro o posición de memoria* contiene el valor correcto, dejando toda la complejidad aritmética al propio binario.

El trabajo permitió poner en práctica conceptos fundamentales de arquitectura de computadoras y sistemas operativos: el formato ELF y el mecanismo PIE/ASLR, el funcionamiento de `ptrace`, la implementación de breakpoints de software con la instrucción `INT3`, y la manipulación directa de registros y memoria de un proceso en ejecución.

Referencias

- [1] Linux man-pages. *ptrace(2) — process trace*. <https://man7.org/linux/man-pages/man2/ptrace.2.html>
- [2] Linux man-pages. *fgets(3)*. <https://man7.org/linux/man-pages/man3/fgets.3p.html>
- [3] Linux man-pages. *sscanf(3)*. <https://man7.org/linux/man-pages/man3/sscanf.3.html>
- [4] Linux man-pages. *strcspn(3)*. <https://man7.org/linux/man-pages/man3/strcspn.3p.html>
- [5] Cloutier, F. *MOVSS — Move or Merge Scalar Single-Precision Floating-Point Value*. <https://www.felixcloutier.com/x86/movss>
- [6] Cloutier, F. *CVTSI2SS — Convert Doubleword Integer to Scalar Single-Precision*. <https://www.felixcloutier.com/x86/cvtsi2ss>
- [7] Cloutier, F. *DIVSS — Divide Scalar Single-Precision Floating-Point Value*. <https://www.felixcloutier.com/x86/divss>
- [8] Cloutier, F. *Jcc — Jump if Condition Is Met*. <https://www.felixcloutier.com/x86/jcc>
- [9] Wikipedia. *TEST (x86 instruction)*. [https://en.wikipedia.org/wiki/TEST_\(x86_instruction\)](https://en.wikipedia.org/wiki/TEST_(x86_instruction))
- [10] Microsoft Docs. *strcmp, wcsncmp, _mbstrcmp*. <https://learn.microsoft.com/es-es/cpp/c-runtime-library/reference/strcmp-wcsncmp-mbstrcmp>
- [11] University of Alaska Fairbanks. *SSE Floating Point in x86 Assembly*. https://www.cs.uaf.edu/2010/fall/cs301/lecture/10_27_sse.html
- [12] YouTube. *ptrace tutorial*. <https://www.youtube.com/watch?v=mXs-iXyA-54>
- [13] YouTube. *Debugging with ptrace*. <https://www.youtube.com/watch?v=VFIZuixNksk>
- [14] Wikipedia. *Executable and Linkable Format*. https://es.wikipedia.org/wiki/Executable_and_Linkable_Format
- [15] ESE DSL. *Que es un kernel en informatica y su importancia para las empresas*. <https://www.esdsl.com/blog/que-es-un-kernel-en-informatica-y-su-importancia-para-las-empresas>
- [16] Wikipedia. *Aleatoriedad en la disposicion del espacio de direcciones*. https://es.wikipedia.org/wiki/Aleatoriedad_en_la_disposici%C3%B3n_del_espacio_de_direcciones
- [17] Linux man-pages. *proc_pid_maps(5)*. https://man7.org/linux/man-pages/man5/proc_pid_maps.5.html
- [18] IBM Docs. *waitpid — Wait for Specific Child Process to End*. <https://www.ibm.com/docs/en/zvm/7.3.0?topic=descriptions-waitpid-wait-specific-child-process-end>

- [19] GeeksforGeeks. *fork() System Call in C*. <https://www.geeksforgeeks.org/c/fork-system-call/>
- [20] Linux man-pages. *execve(2)*. <https://man7.org/linux/man-pages/man2/execve.2.html>